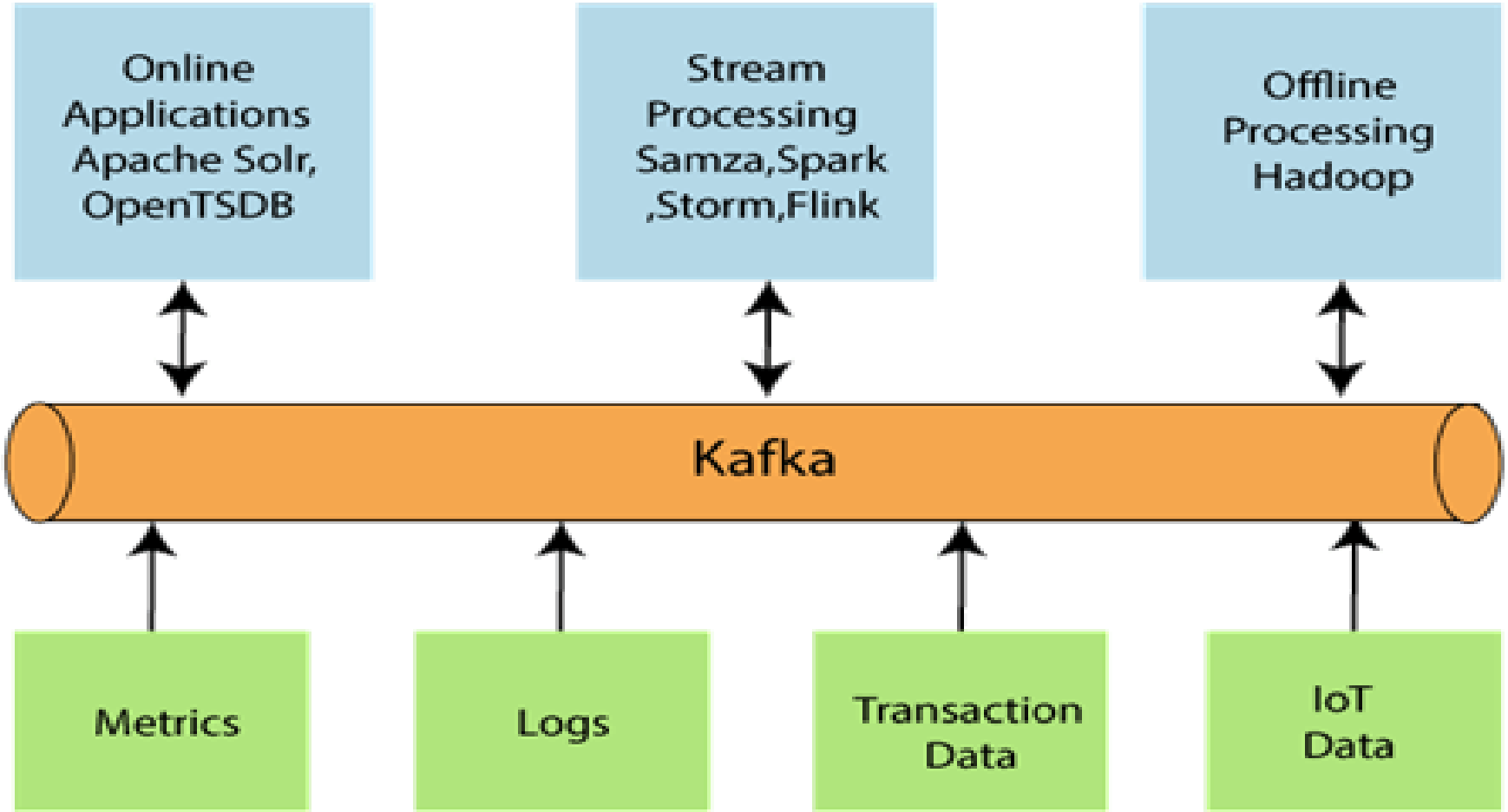
Kafka

Introduction



contd..

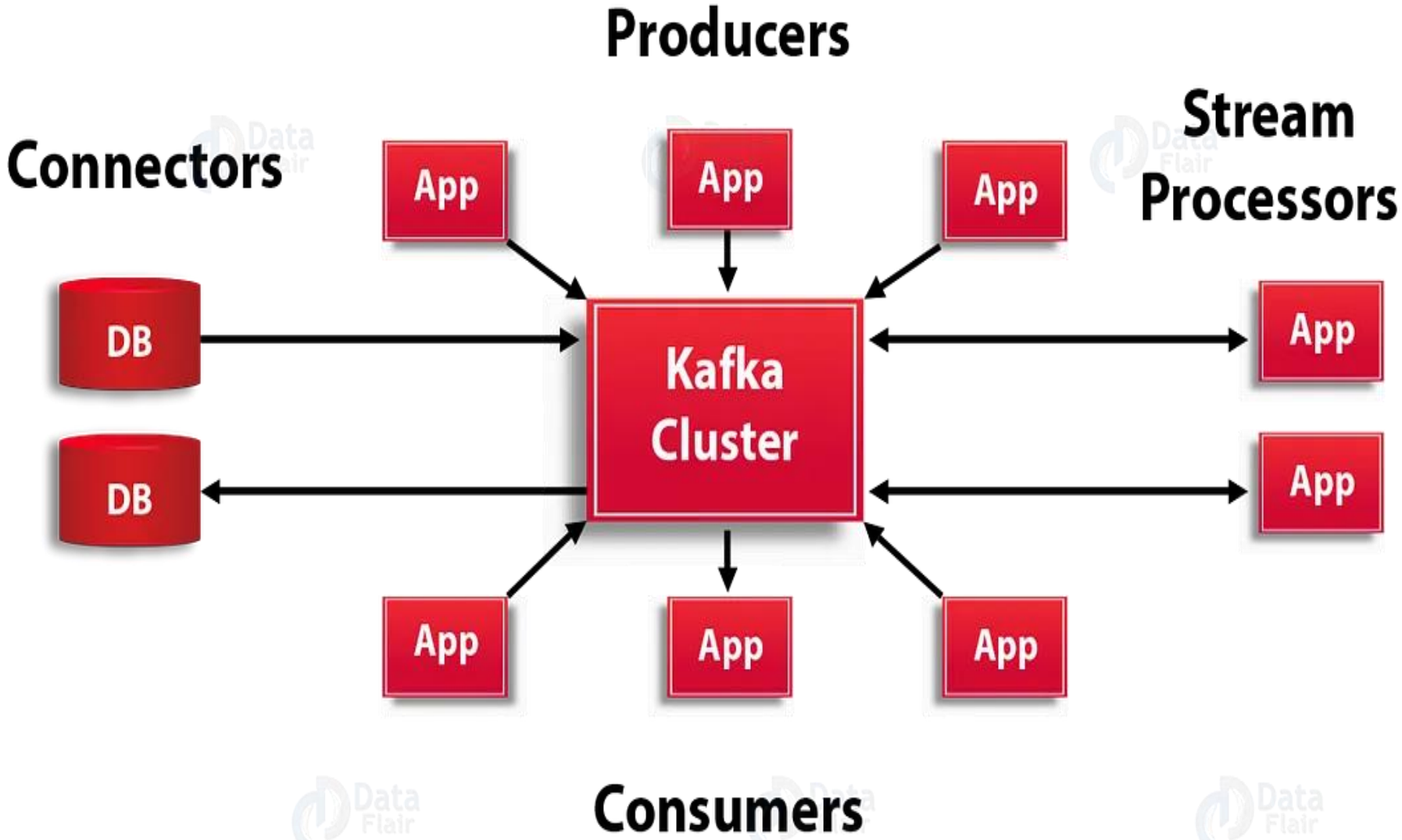
- Kafka is **distributed, event streaming platform.** (**distributed** as Kafka can live on multiple machines).
- **Kafka** was conceived as **messaging queue** in **2011** by **LinkedIn** as a streaming platform, a system that lets to publish and subscribe streams of data, store them, and process events (i.e. messages) of something that happened (sent as a key/value pair).
- Every enterprise application creates data viz..
  a. Log messages
  b. Metrics
  c. User activity
  d. Outgoing messages, etc
- **Moving & Managing** this data is as important as data itself – treat data as continually evolving and ever-growing stream, **build a data architecture** – oriented around that idea.
- Messages are processed as **stream of events –** no schedule, handled as they arrive.

contd..

- **Kafka** can:
  a. ingest data in real-time
  b. manipulate, store and retrieve
  c. route the event streams to different destination
- **Kafka** has differences from traditional messaging systems –

1. Works as **distributed system** that runs as **cluster** and can scale to handle all the applications in even the most massive of companies. Has **central platform** that can scale elastically to handle all the streams of data in a company.
2. Is a true **storage system** built to **store data**. It is advantageous in using it as connecting layer as it provides real delivery guarantees — its data is replicated, persistent, can be kept around as needed.
3. Stream processing raises the abstraction level while messaging system just hand out messages. Kafka stream processing capabilities computes derived streams and datasets dynamically off of streams with less code.

Kafka Components



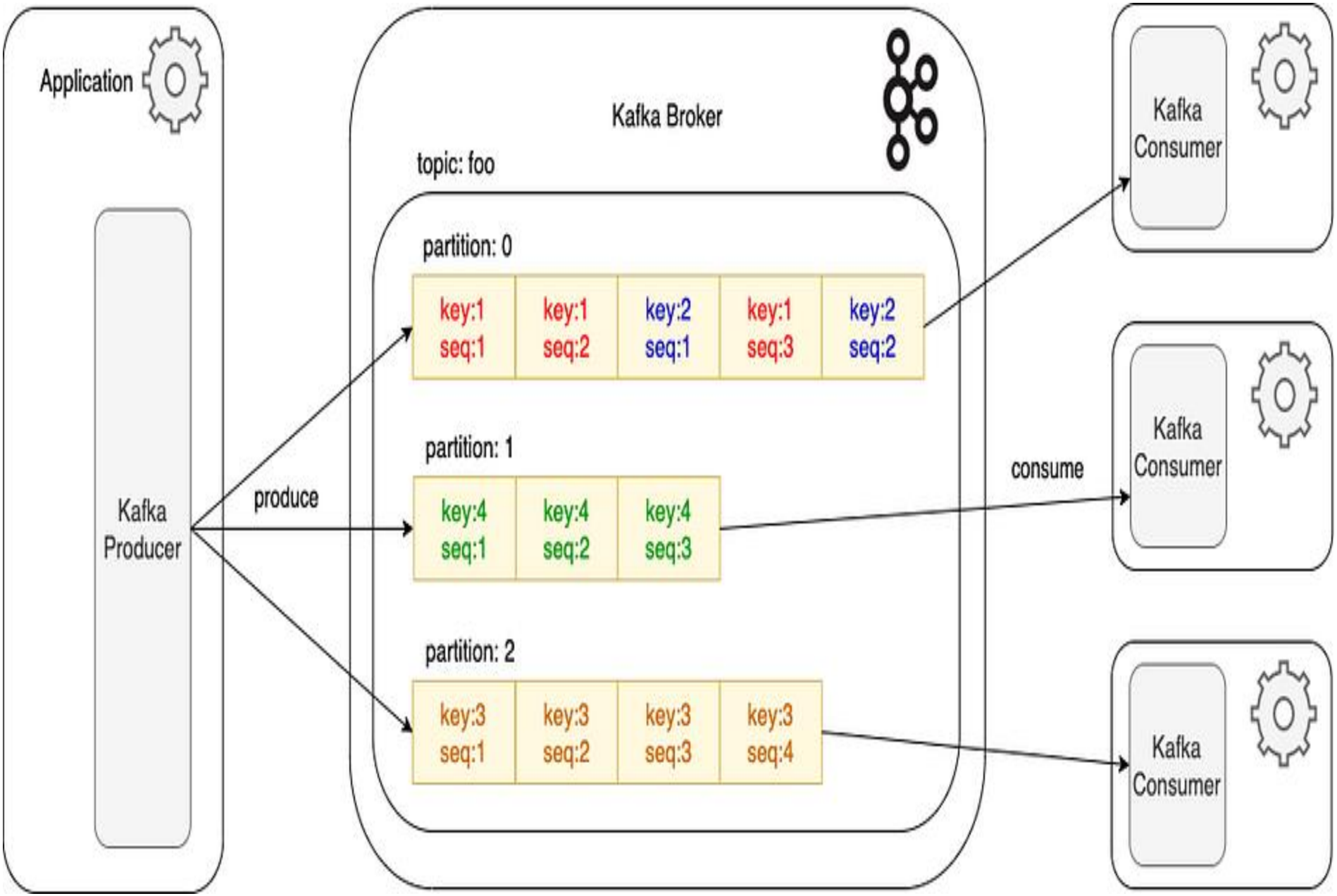
contd..

- **Events/Messages**: the messages of something that occurred. Both events and messages are stored as "**records**" within Kafka topics, consisting of a key, value, and optional headers.
- **Message Queue** – service that provides to buffer user's flow of data.
  a. An application sends messages in, the messages get buffered, and another service reads those messages.
  b. The data is temporarily stored and processed later.
- **Broker –** the Kafka servers that handle the data.
- **Producers** – a client application that **creates events** and **publishes** to **Kafka topic**. Producer pushes **message** to broker on a given topic. Producer is the source of data.
  - ✓ Producer **does not send** messages **directly** to consumers rather **pushes messages** to Kafka Server or Broker.
  - ✓ **Multiple producers** can send a message to the same Kafka topic or different Kafka topics.

contd..

- **Consumers**: a client application that subscribes to a topic and reads the data stored in the broker.
  - ✓ The consumer acts as the Receiver.
  - ✓ Consumer does not consume or receive a message directly from Kafka Producer.
  - ✓ Consumer can request a message from the Kafka broker. If the Kafka Consumer will have enough permissions, then it gets a message from the Kafka Broker.
- **Kafka cluster** contains **one** or **more brokers** which store the messages received from Producer to a Kafka topic.
  - ✓ Kafka is distributed system, so it has a cluster having a group of servers called brokers.
  - ✓ Kafka cluster, has one or more Kafka brokers.

contd..


Application
Kafka Producer
produce
Kafka Broker
topic: foo
partition: 0
key:1 seq:1
key:1 seq:2
key:2 seq:1
key:1 seq:3
key:2 seq:2
partition: 1
key:4 seq:1
key:4 seq:2
key:4 seq:3
partition: 2
key:3 seq:1
key:3 seq:2
key:3 seq:3
key:3 seq:4
consume
Kafka Consumer
Kafka Consumer
Kafka Consumer

contd..

- **Topic**: a collection of messages of the **same type**.
  - ✓ The producers send messages to the **topics**, the consumers subscribe to the topics and receive the messages.
  - ✓ Kafka Topic is a unique name given to a data stream or message stream.
  - ✓ A consumer wants to consume a message from Broker, but from which message stream? There can be multiple different message streams on the same Broker, coming from different Kafka producers.
  - ✓ Topic is a unique identity of the message stream.
  - ✓ The Producer sends a message to a unique name which is called the topic for that message stream. Multiple producers can also send to the same topic.
  - ✓ If any consumer wants to consume the message, then it subscribes to the topic present in Kafka Broker. Now all the messages coming to that topic will be delivered to the consumer.

contd..

- **Partition –** a fundamental, ordered, and immutable sequence of records (messages) that makes up a portion of a Kafka topic, acting as the primary unit of parallelism and storage.
  - ✓ By splitting topics into multiple partitions distributed across different brokers, Kafka achieves high throughput, linear scalability for producers/consumers, and fault tolerance through replication.
  - ✓ Use case – consider there is huge volume of data, and it is very challenging for the broker to store data on a single machine. Kafka topic can be broken into partitions and distribute the partitions on a different machine to store.
  - ✓ Based on the use case and data volume, we can decide the number of partitions for a topic during Kafka topic creation.

contd..

- **Offsets –** a sequence number, assigned to each message in each partition of a Kafka topic. This sequence number is called Offset.
  - ✓ A number is assigned to that message upon arrival in a partition. Different partitions have different offsets in a partition.
  - ✓ The offset number is always local to the topic partition. There is not any offset that is global to the topic or each partition of the topic.
  - ✓ Initially, the offset pointer points to the first message. As soon as the consumer reads that message, the offset pointer moves to the next message and so on in the sequence.
  - ✓ So, the combination of topic name, partition number, and offset number is a unique identity for any message. In other words, you can find any message based on the below three components.

contd..

KAFKA COMPONENTS

- Topics
- Zookeeper
- Brokers
- Producers
- Consumers

Event & Message

- **"Event"** signifies a notification about something that has already happened, representing a state change, while **"message"** is broader term for any data being sent, which could include requests or updates, not necessarily tied to a specific past occurrence.
- **Event & Message** both are stored as **data units** within **Kafka topics**. The key distinction lies in the intent behind the data transmission.
- **Event focus:** An event indicates a fact or occurrence that has already taken place, essentially a notification about state change in a system.
- **Message focus:** A message is a more general term for any data being sent, which could represent a request, an update, or even an event notification.
- **Consumption:** Consumers can subscribe to specific topics to receive both events and messages, depending on data they need to process.

Kafka Architecture

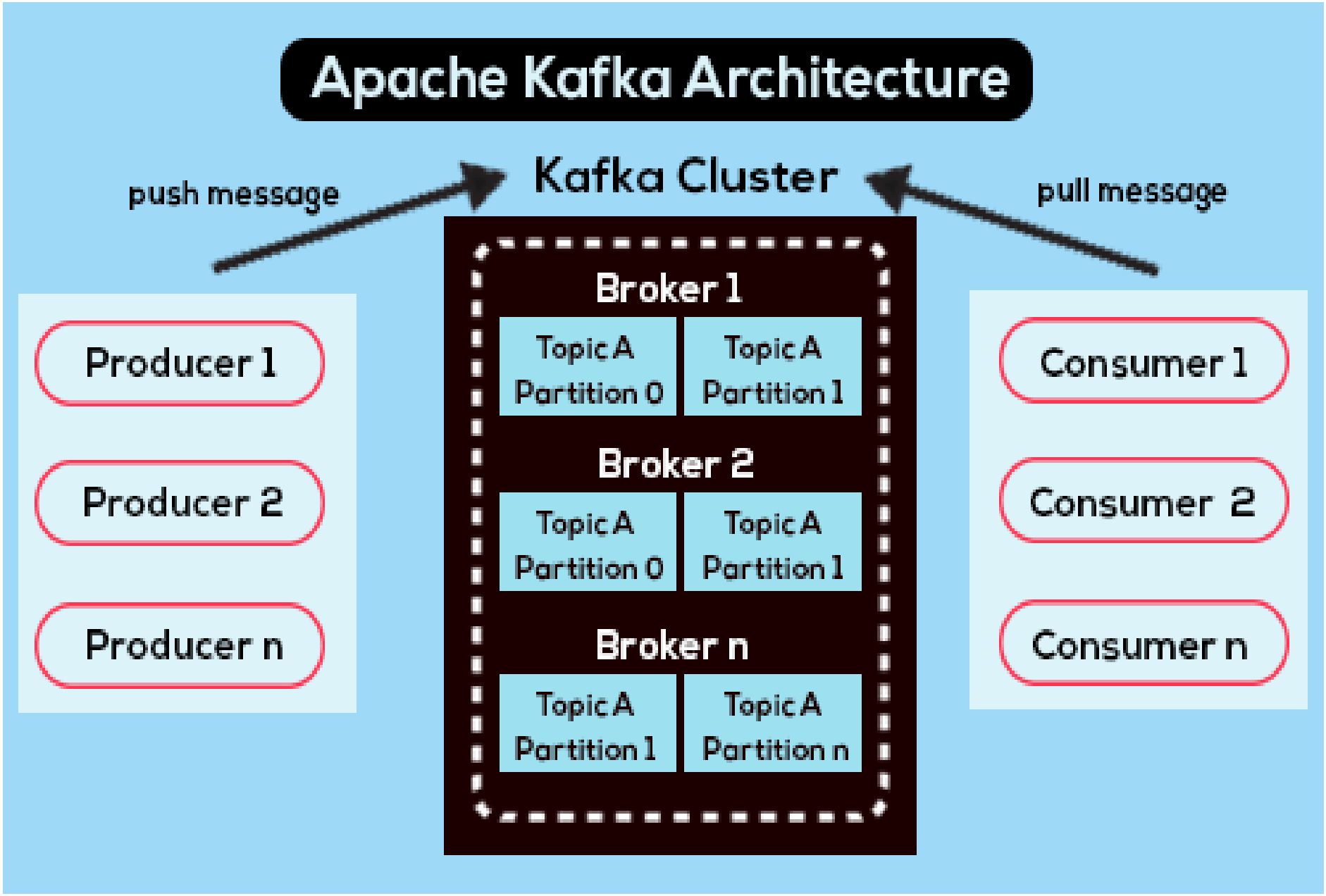
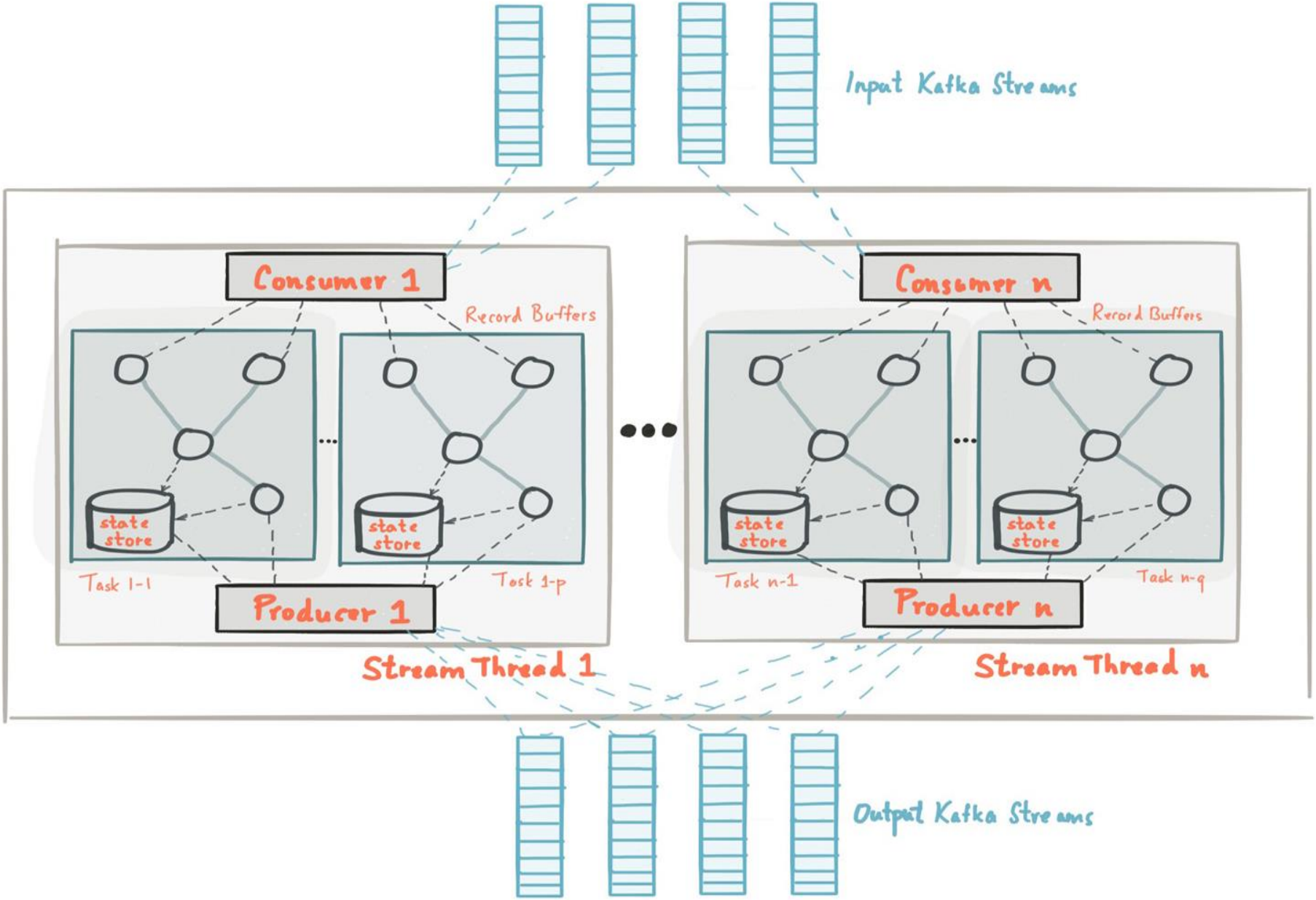


Diagram – II


Input Kafka Streams
Consumer 1
Record Buffers
state store
state store
Task 1-1
Task 1-p
Producer 1
Stream Thread 1
Consumer n
Record Buffers
state store
state store
Task n-1
Task n-q
Producer n
Stream Thread n
Output Kafka Streams

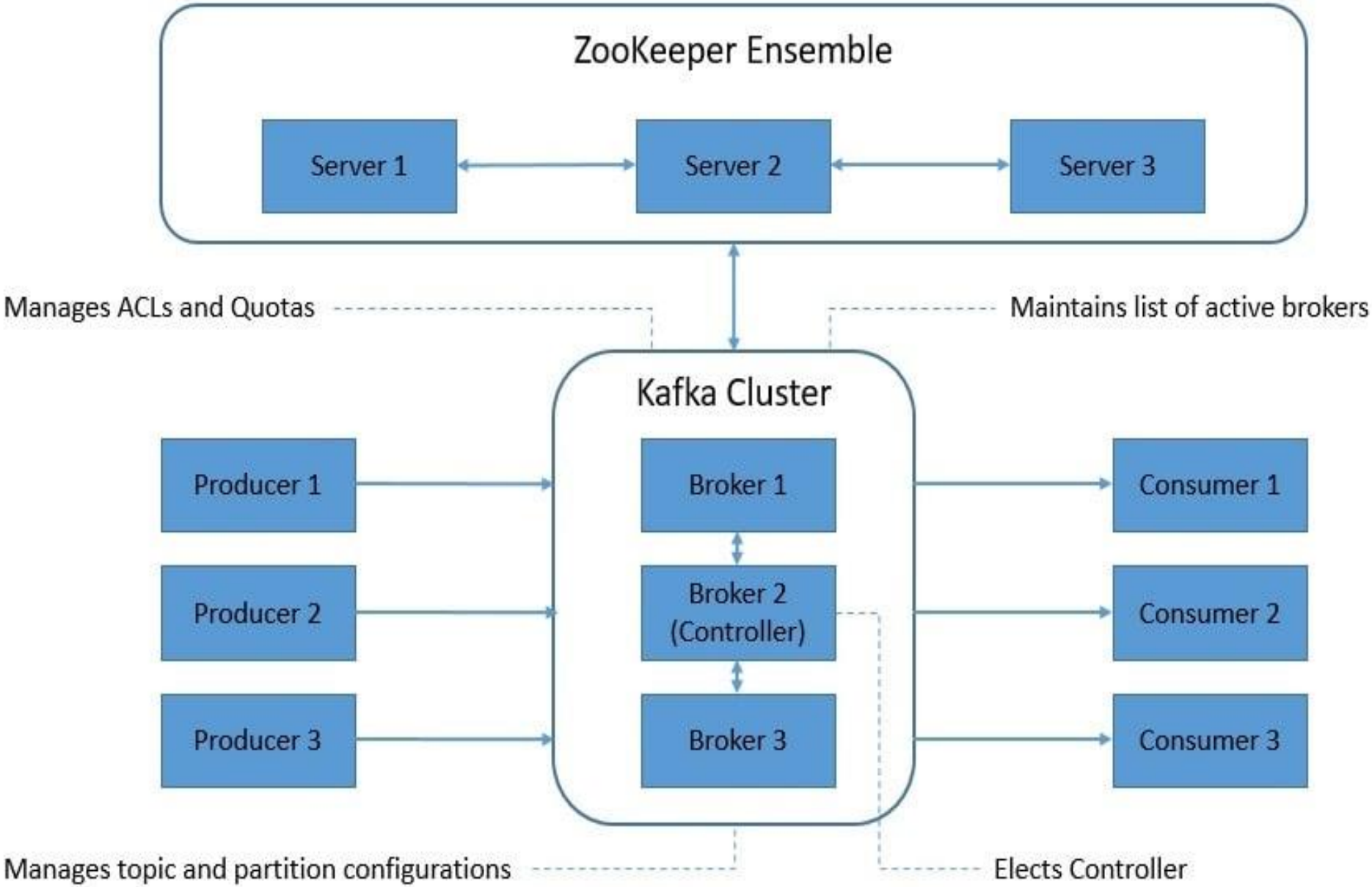
contd..

- Apache ZooKeeper is a centralized, highly reliable coordination service used by Apache Kafka to manage cluster metadata, broker membership, and partition leadership.
- Often described as the "central nervous system" of a Kafka cluster, it ensures consistency and synchronizes state among nodes.
- ZooKeeper acts as the source of truth, enabling fault tolerance and configuration management, although it is being replaced by KRaft in newer Kafka versions.
- In versions 4.0 onwards **Controller Quorum** replaces ZooKeeper by maintaining cluster metadata internally using the Raft consensus protocol. Instead of an external dependency, dedicated controller nodes or combined broker-controller nodes manage leader elections and partition assignments. This simplifies deployment to a single, unified process while significantly improving failover times and scalability to millions of partitions.

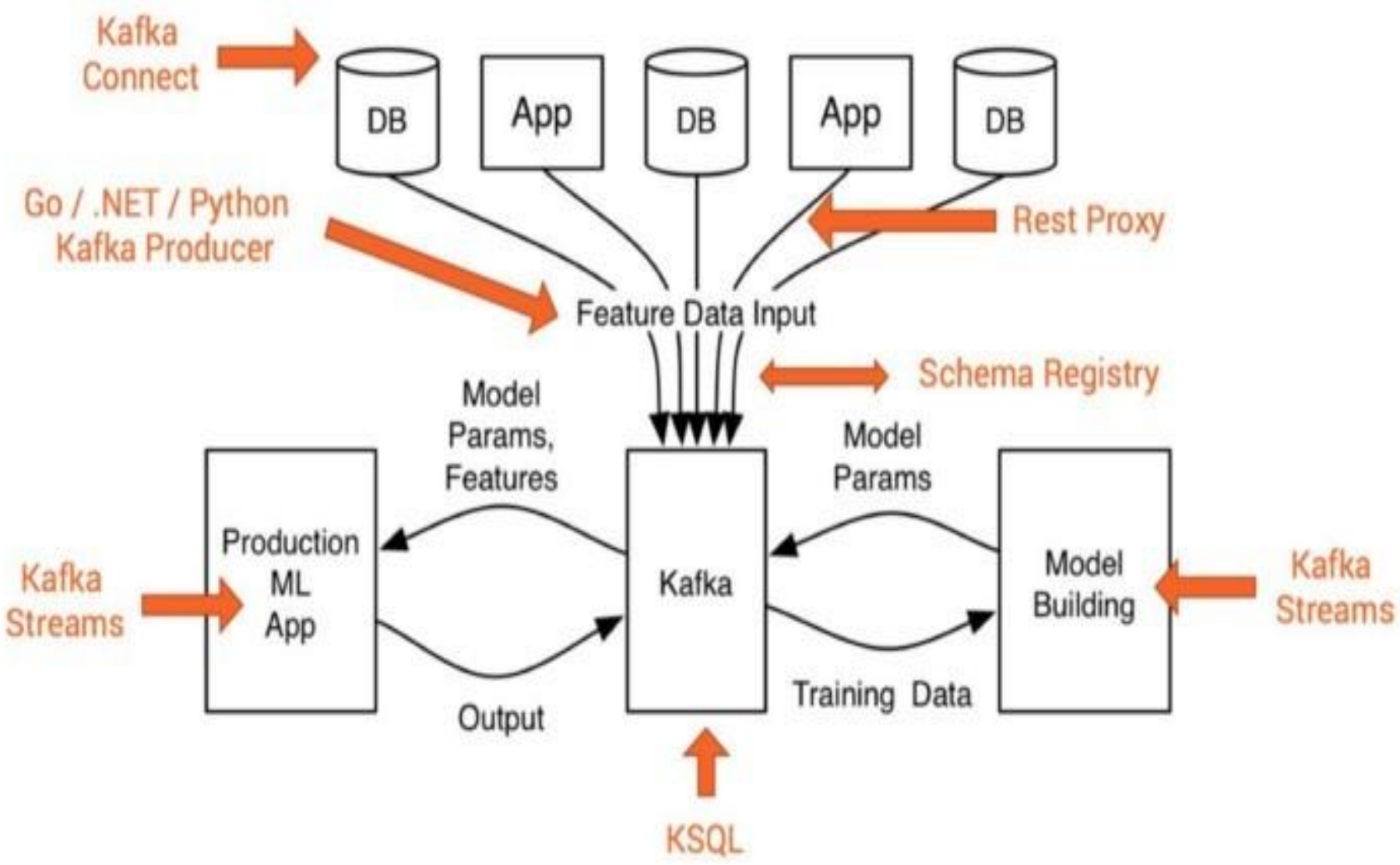
contd..

- **KRaft Architecture Components**
- ✓ **Controller Quorum:** A set of nodes (typically 3 or 5) that run the Raft algorithm to maintain the cluster state.
- ✓ **Broker Nodes:** Handle data storage and client requests. In many setups, these also serve as controllers (process.roles=broker,controller).
- ✓ **Metadata Log (__cluster_metadata):** The internal Kafka topic where the Raft quorum stores all cluster information (topics, partitions, configurations) as the single source of truth.
- ✓ **Producers & Consumers:** Connect directly to brokers without needing to interact with a separate ZooKeeper ensemble.

Zookeeper based diagram



contd..


Kafka Connect
DB
App
DB
App
DB
Go / .NET / Python Kafka Producer
Rest Proxy
Feature Data Input
Schema Registry
Model Params, Features
Model Params
Kafka Streams
Production ML App
Kafka
Model Building
Kafka Streams
Output
Training Data
KSQL

- Example:
- **Event:**
- A user placing an order on an online store would be considered an "event" as it signifies a completed action.
- **Message:**
- A message could be the actual order details sent to the order processing system, which includes information about the event (the placed order).

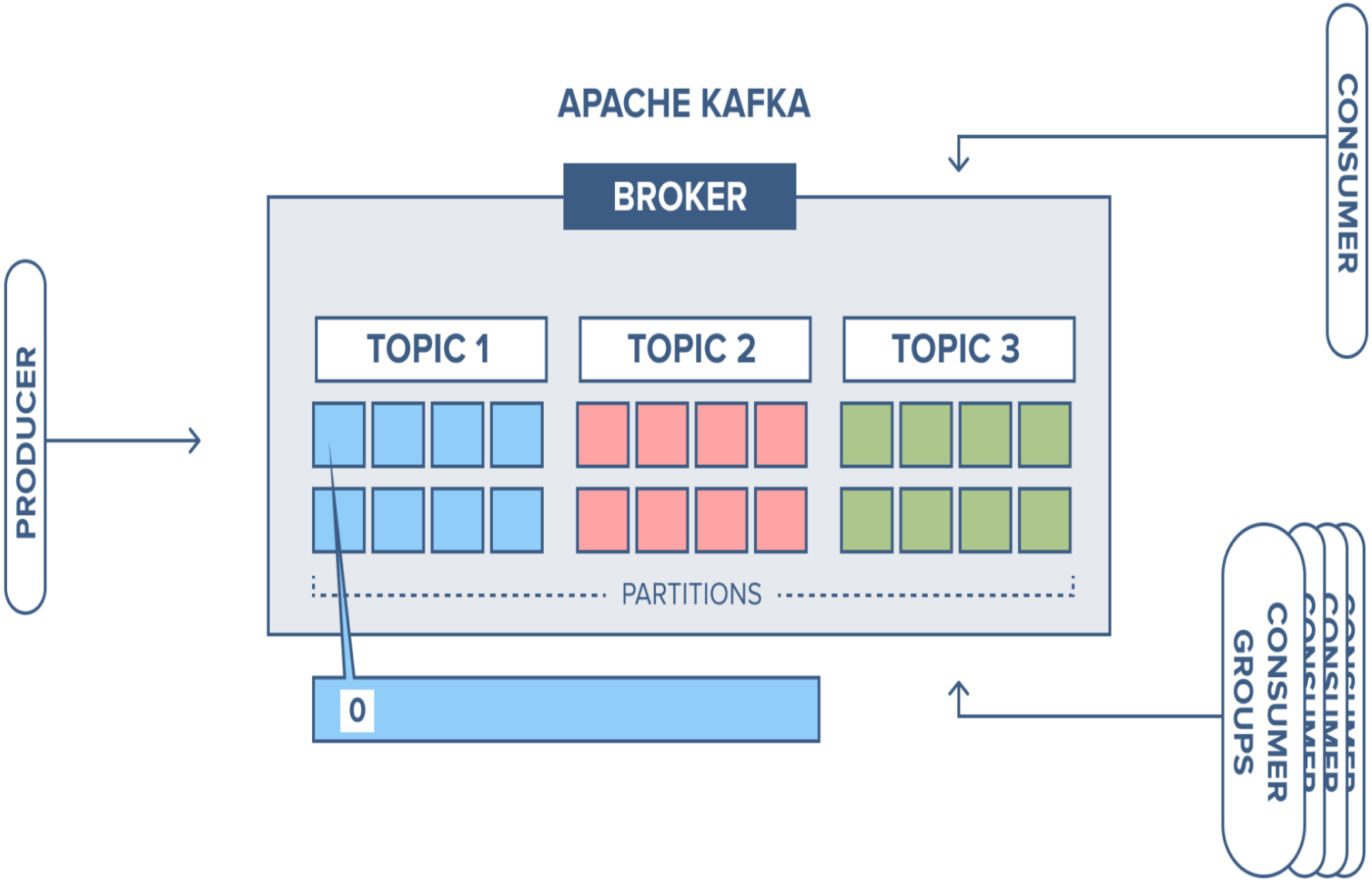
contd..

- **Events** (i.e messages) are the **basic unit** of Kafka streaming. They can be anything that is to be streamed from one service to another.
- **Events** are stored as byte arrays. Events are made of **key/value** and **timestamp**. **Key** is optional, its **not unique** and it can be sent as **null**. **Timestamp** can also be sent as **null**. If that's the case, Kafka will automatically add one.
- Even if the key is not mandatory, if you care about the order of consuming the messages, it might be a good idea to send the events to the same partition using the same key.
- There is a maximum size of the message that you can send. Finally, you can set different parameters for delivery guarantees: exactly 1, at least 1, at most 1 (i.e. the message can be dropped).
- Example: **Event:** A user placing an order on an online store would be considered an "event" as it signifies a completed action.
- **Message:** A message could be the actual order details sent to the order processing system, which includes information about the event (the placed order).

contd..

- Kafka keys and timestamps are not unique because their primary purpose is **data ordering and partitioning**, not record identification. Keys (e.g., user_id) group related events into the same partition, while timestamps are often generated by producers or brokers, allowing multiple events to share the same time, particularly when updates occur frequently.
- Apache Kafka is a software where topics (A topic might be a category) can be defined and further processed. Applications may connect to this system and transfer a message onto the topic.
- A Kafka cluster usually consists of one or more servers called as kafka brokers, which are running Kafka over them.
- **Producers** are processes that publish data (push messages over trigger) into Kafka topics within the specified broker.
- A consumer of topics pulls messages off a Kafka topic.
-

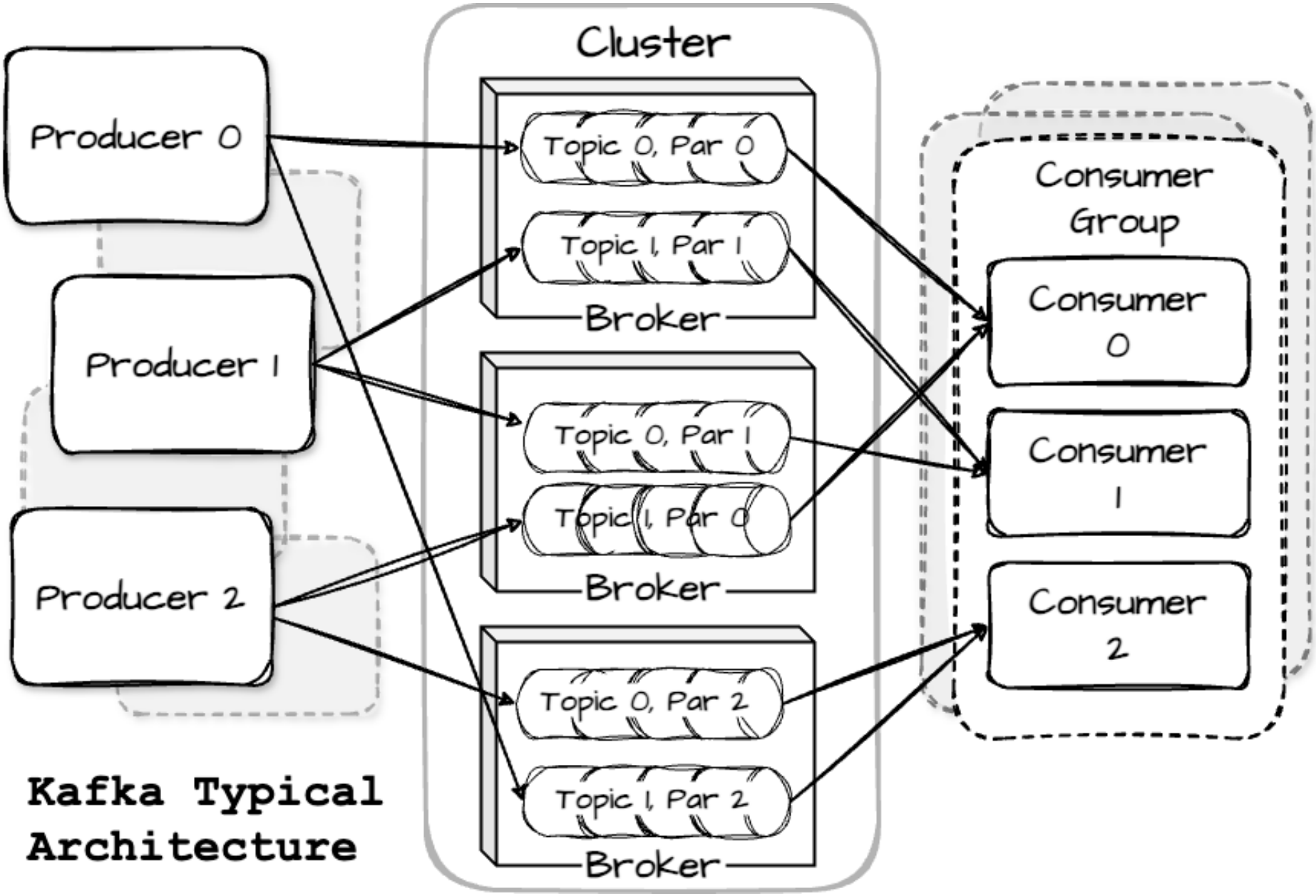
Kafka Topic


CONSUMER
CONSUMER
CONSUMER
CONSUMER
CONSUMER
GROUPS
APACHE KAFKA
BROKER
TOPIC 1
TOPIC 2
TOPIC 3
PARTITIONS
0
PRODUCER

contd..

- **Topic** is basically a **category** or a **feed name** to which messages are stored and published during operations. Messages are mostly byte arrays that can store any object in any format.
- **Topic** is identified by its name, which depends on the user's choice.
- **Topics** in Kafka are similar to tables in the database, but not containing all constraints. Any **number** of topics can be created.
- Kafka **messages** are organized into **topics**. If you wish to send a message you send it to a specific topic and if you wish to read a message you read it from a specific topic.
- Any object can be stored as byte array.
- A producer publishes data to the topics, and a consumer reads that data from the topic by subscribing it.

contd..

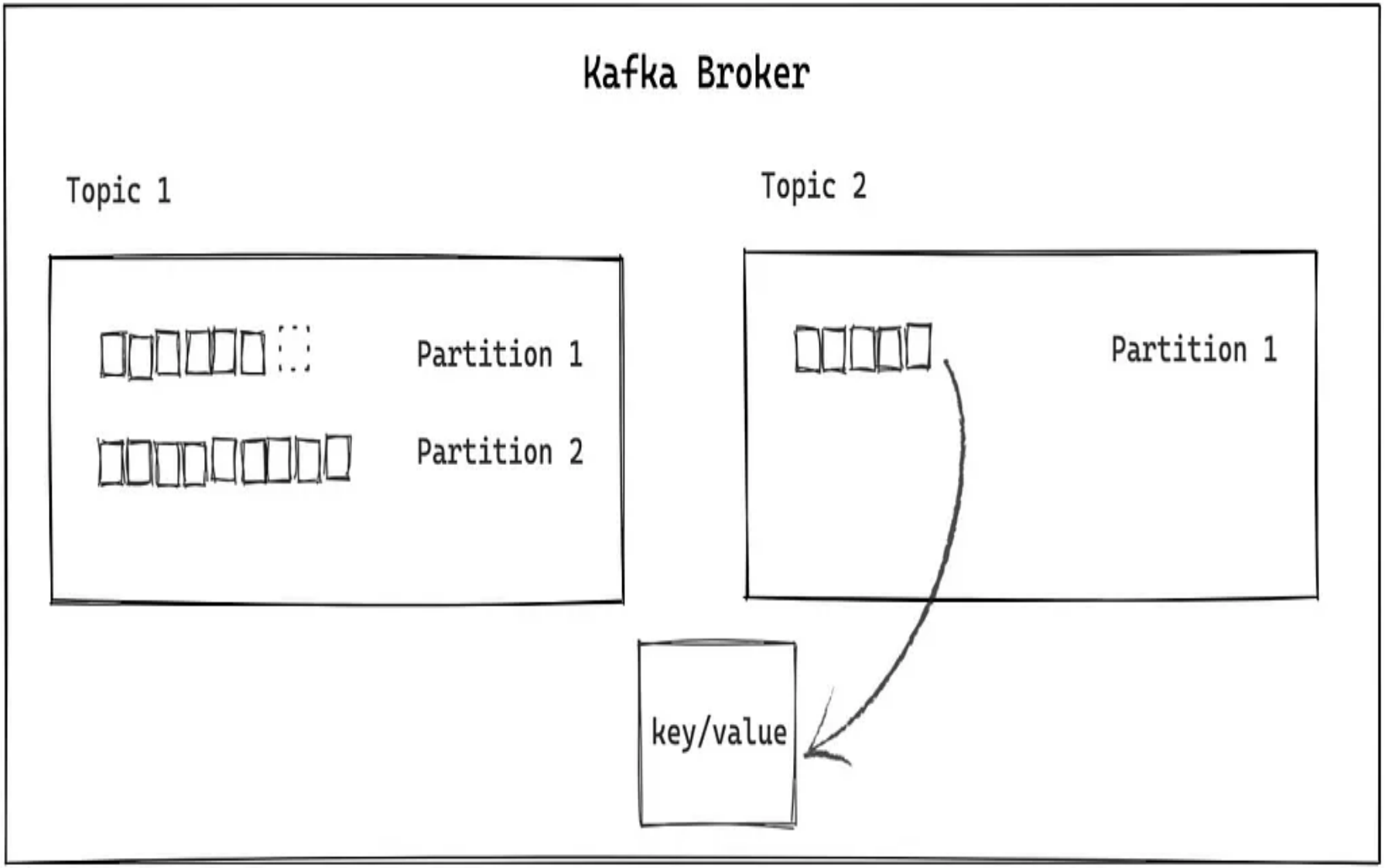


Kafka Typical Architecture

Partitions

- **Topic** is split into several parts which are known as the **partitions** of the **topic.**
- These partitions are separated in an order.
- The data content gets stored in the partitions within the topic. Therefore, while creating a topic, the number of partitions(the number is arbitrary) is specified.
- Each message gets stored into partitions with an **incremental-id** known as its Offset value.
- The order of the **offset value** is guaranteed within the partition only and not across the partition. The offsets for a partition are infinite.

contd..


Kafka Broker
Topic 1
Partition 1
Partition 2
Topic 2
Partition 1
key/value

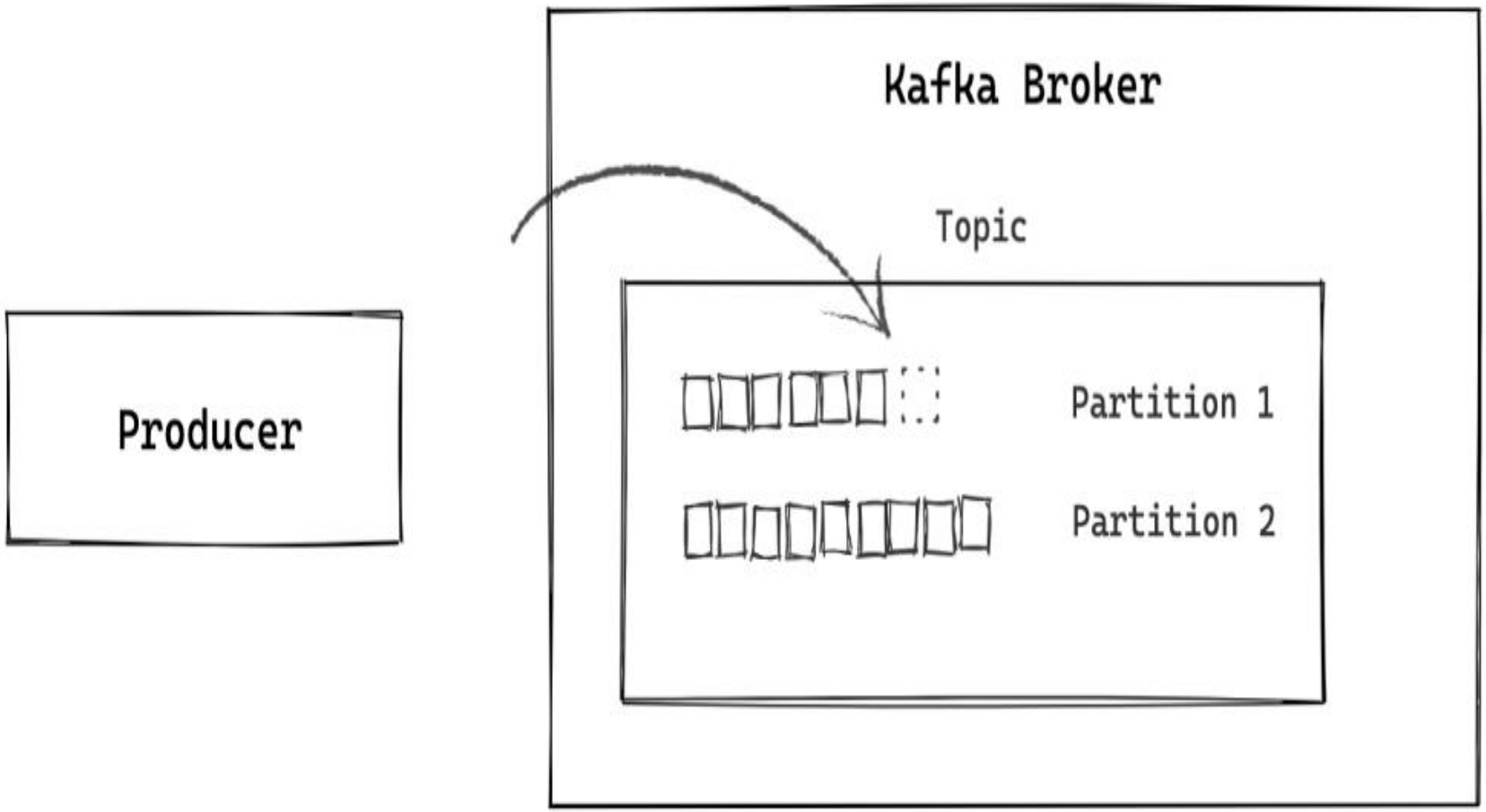
Producers in Kafka

- **Producer** is the service that produces (i.e. sends) messages to Kafka.
- The producer publishes the messages to a topic. In which partition will the message fall?
- The partition is determined with this formula:

$$message_partition = hash(message_key) \% partition_number$$

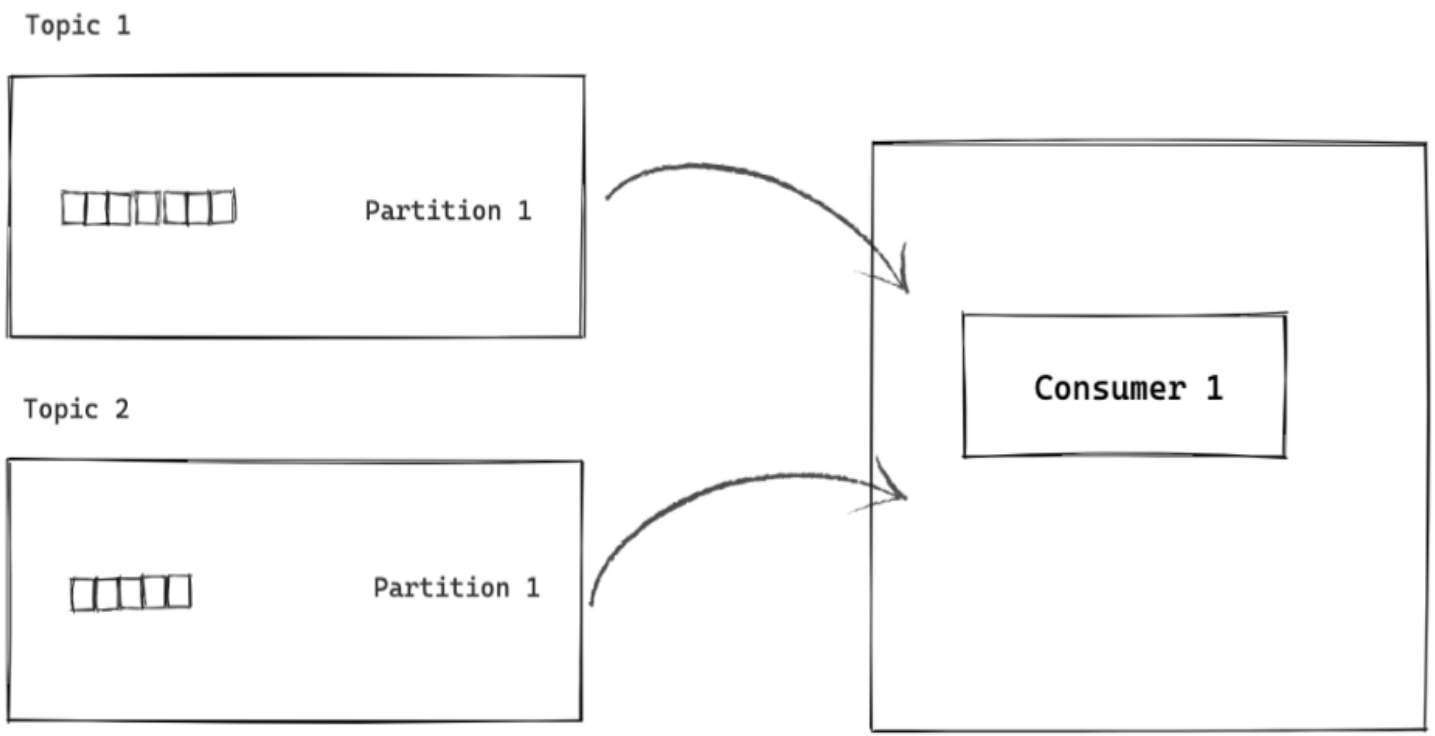
- Kafka applies a hash function to the key of the message and then takes the module with the number of partitions.
- If a message doesn't have a key specified, Kafka will automatically try to make the partition balanced among themselves.

contd..



contd..

- Consumers are the services that consume (i.e. receive) the messages. A consumer can subscribe to one or more topics.



contd..

- **Data Ecosystem:** Several applications that use Apache Kafka forms an ecosystem. This ecosystem is built for data processing. It takes inputs in the form of applications that create data, and outputs are defined in the form of metrics, reports, etc.
- **Kafka Cluster:** A Kafka cluster is a system that comprises of different brokers, topics, and their respective partitions. Data is written to the topic within the cluster and read by the cluster itself.
- **Producers:** A producer sends or writes data/messages to the topic within the cluster. In order to store a huge amount of data, different producers within an application send data to the Kafka cluster.
- **Consumers:** A consumer is the one that reads or consumes messages from the Kafka cluster. There can be several consumers consuming different types of data form the cluster. The beauty of Kafka is that each consumer knows from where it needs to consume the data.

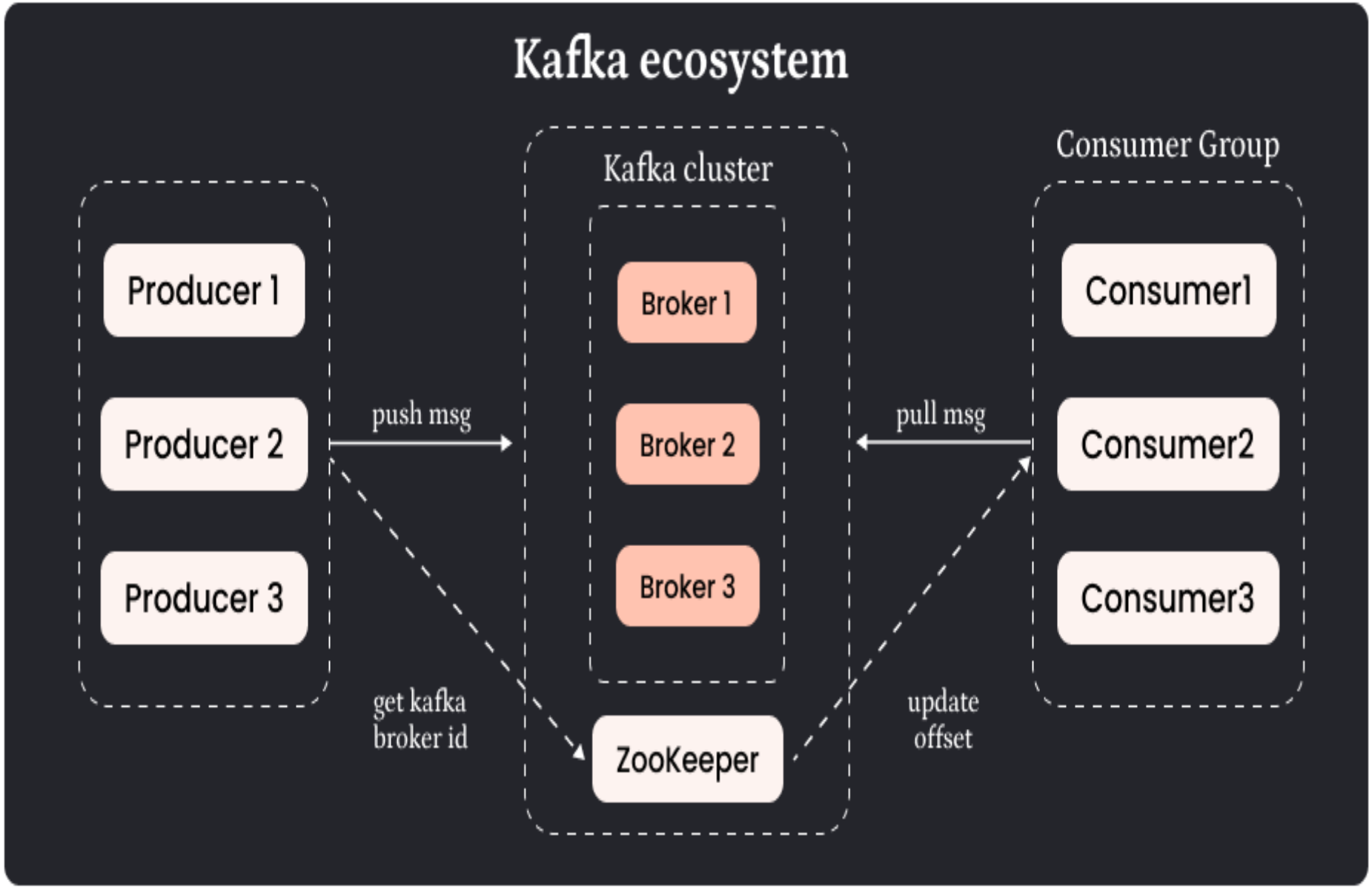
contd..

- **Brokers:** A Kafka server is known as a broker. A broker is a bridge between producers and consumers. If a producer wishes to write data to the cluster, it is sent to the Kafka server. All brokers lie within a Kafka cluster itself. Also, there can be multiple brokers.
- **Topics:** It is a common name or a heading given to represent a similar type of data. In Apache Kafka, there can be multiple topics in a cluster. Each topic specifies different types of messages.
- **Partitions:** The data or message is divided into small subparts, known as partitions. Each partition carries data within it having an **offset** value. The data is always written in a sequential manner. We can have an infinite number of partitions with infinite offset values. However, it is not guaranteed that to which partition the message will be written.

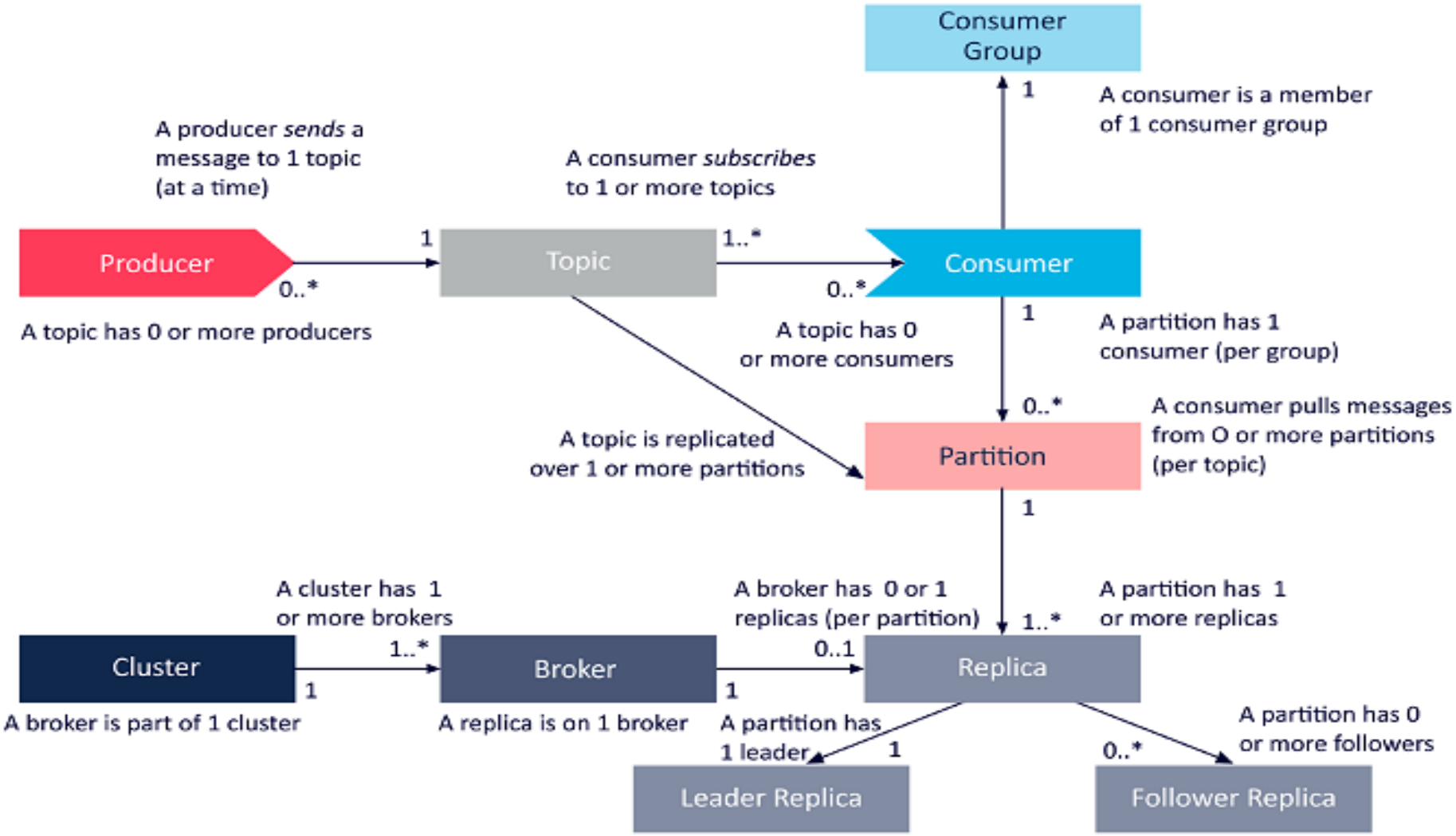
Zookeeper

- ZooKeeper is used to store information about the Kafka cluster and details of the consumer clients.
- It manages brokers by maintaining a list of them.
- ZooKeeper is responsible for choosing a leader for the partitions.
- If any changes like a broker die, new topics, etc., occurs, the ZooKeeper sends notifications to Apache Kafka.
- ZooKeeper is designed to operate with an odd number of Kafka servers.
- Zookeeper has a leader server that handles all the writes, and rest of the servers are the followers who handle all the reads.
- A user does not directly interact with the Zookeeper, but via brokers. No Kafka server can run without a zookeeper server. It is mandatory to run the zookeeper server.

Diagram


Kafka ecosystem
Kafka cluster
Consumer Group
Producer 1
Producer 2
Producer 3
Broker 1
Broker 2
Broker 3
Consumer1
Consumer2
Consumer3
push msg
pull msg
get kafka
broker id
ZooKeeper
update
offset

Kafka flow


Consumer Group
1
A consumer is a member of 1 consumer group
A producer *sends* a message to 1 topic (at a time)
A consumer *subscribes* to 1 or more topics
Producer
1
Topic
1..*
Consumer
0..*
0..*
A topic has 0 or more producers
A topic has 0 or more consumers
1
A partition has 1 consumer (per group)
0..*
A consumer pulls messages from O or more partitions (per topic)
A topic is replicated over 1 or more partitions
Partition
1
A cluster has 1 or more brokers
A broker has 0 or 1 replicas (per partition)
A partition has 1 or more replicas
1..*
Cluster
1..*
Broker
0..1
Replica
1
1
A broker is part of 1 cluster
A replica is on 1 broker
A partition has 1 leader
1
A partition has 0 or more followers
0..*
Leader Replica
Follower Replica

Kafka Throughput

- Kafka works on event streaming model i.e. log based model.
- Kafka keeps all messages for set amount of time. Long term persistence. Designed for data analytics.
- High tp – rate at which messages are processed. Metric that tells amount of data system can handle in a given interval of time.
- Kafka can handle large amount of data in

Kafka internal working

- **Kafka does NOT associate topics directly with UUID**
- That UUID is a **Cluster ID.** UUID is for **cluster**, not for topics
- When user runs :
  - ✓ kafka-storage.bat format -t <UUID> ...
- Kafka stores this ID inside:
  - ✓ logs/meta.properties
- It looks like:
  - ✓ cluster.id=tgWlxgKWQ5qf4iAb8nLwJw
- This tells Kafka:
  - ✓ "This storage belongs to THIS cluster"

contd..

- Topics are NOT linked to UUID directly
- Instead: Topics are stored inside the same log directory. Example -
- logs/
  meta.properties ← contains cluster.id
  test-topic-0/ ← your topic partition
  product-topic-0/
- How Kafka connects everything
- Cluster (UUID)
  ↓
  Broker (your Kafka server)
  ↓
  Log directory (logs/)
  ↓
  Topics (folders inside logs/)
- UUID → identifies the cluster
- Topics → stored inside that cluster's storage

contd..

- The command – kafka-topics.bat --create --topic test-topic ...
- Kafka uses running broker (already tied to cluster UUID)
- Creates folder: logs/test-topic-0/
- Registers metadata in: __cluster_metadata
- Kafka server already knows its cluster (via UUID)
- So when a topic is created – It automatically belongs to that cluster
  No need to "attach" topic to UUID manually .
- E.g. UUID = University ID
- Kafka Server = Campus
- Topics = Classrooms

When you create a classroom, it automatically belongs to that campus. You don't assign campus ID to each classroom manually

contd..

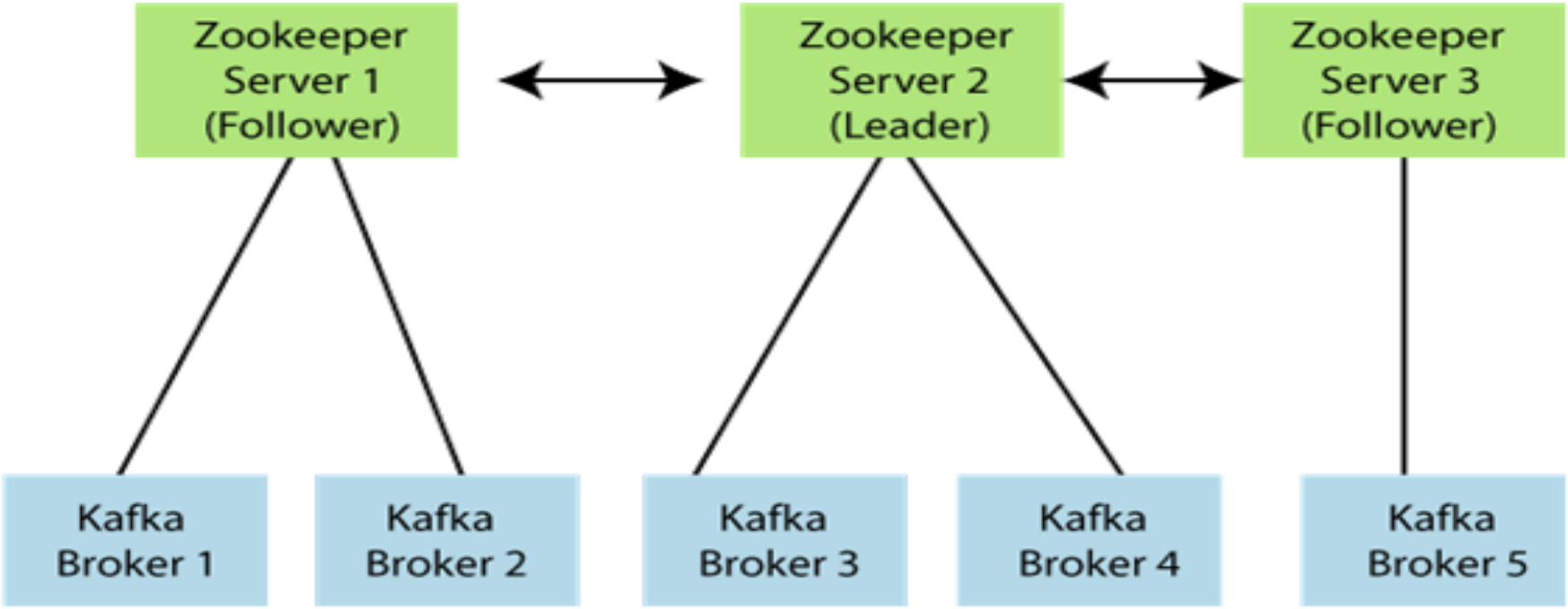
- UUID becomes important when:
- User reuse old logs with different UUID You'll get error like: - Invalid cluster.id. Because: Logs say: cluster A You start with: cluster B
- Reset scenario
  - ✓ If user want new cluster:
  - ✓ rmdir /s /q logs
    kafka-storage format ...

- Kafka server does NOT check “topic → UUID mapping”
  👉 Instead:
- ✓ UUID identifies the cluster
  ✓ Broker uses that cluster
  ✓ Topics are created inside that cluster automatically

- A basic Kafka program example would involve creating a producer to send messages to a specific topic and a consumer to read messages from that topic, often implemented in Java using the Kafka client library; for instance, a simple scenario could be a web application sending user login events to a "user-login" topic in Kafka, which a separate service then consumes to process further actions like sending welcome emails.

- Key elements in a Kafka program:
- **Topic:** A logical category where messages are published and subscribed to.
- **Producer:** An application that sends messages to a topic.
- **Consumer:** An application that reads messages from a topic.
- **Partitioning:** A mechanism to distribute messages across multiple brokers for scalability.

contd..



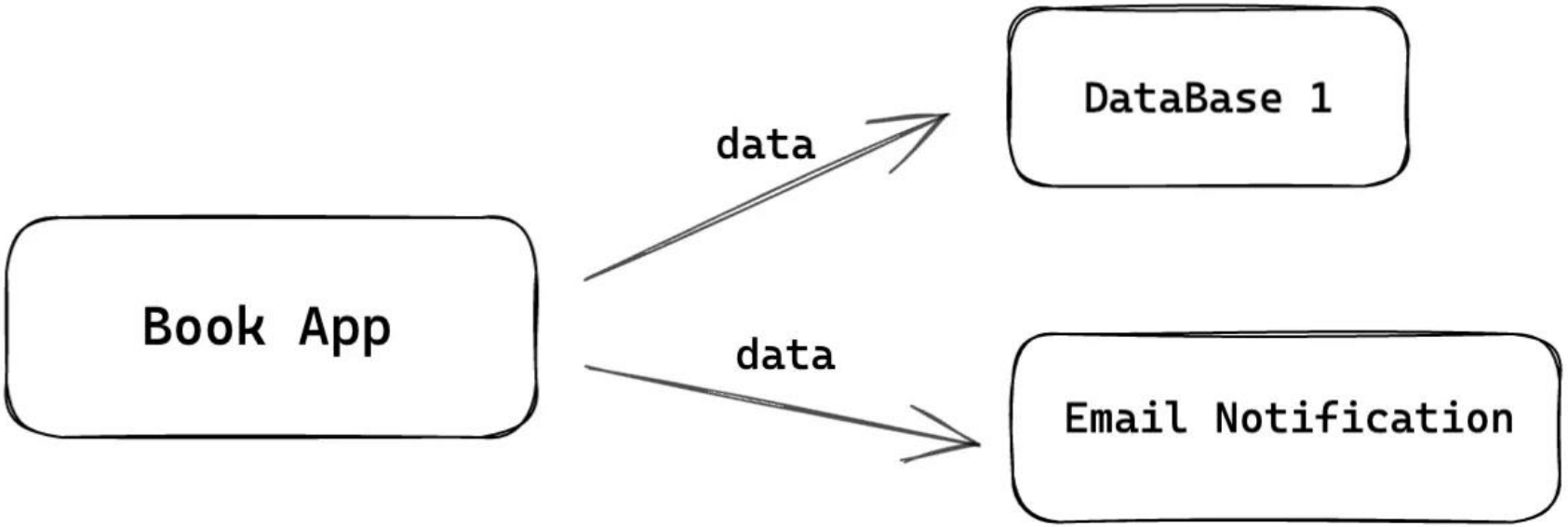
There are three zookeeper servers where server 2 is the leader, and the other two are chosen as its followers.

The five brokers are connected to these servers. Automatically, the Kafka cluster will come to know when brokers are down, more topics are added, etc..

1. Every byte of data has a story to tell, something of importance that will inform the next thing to be done. In order to know what that is, we need to get the data from where it is created to where it can be analyzed. We see this every day on websites like Amazon, where our clicks on items of interest to us are turned into recommendations that are shown to us a little later.

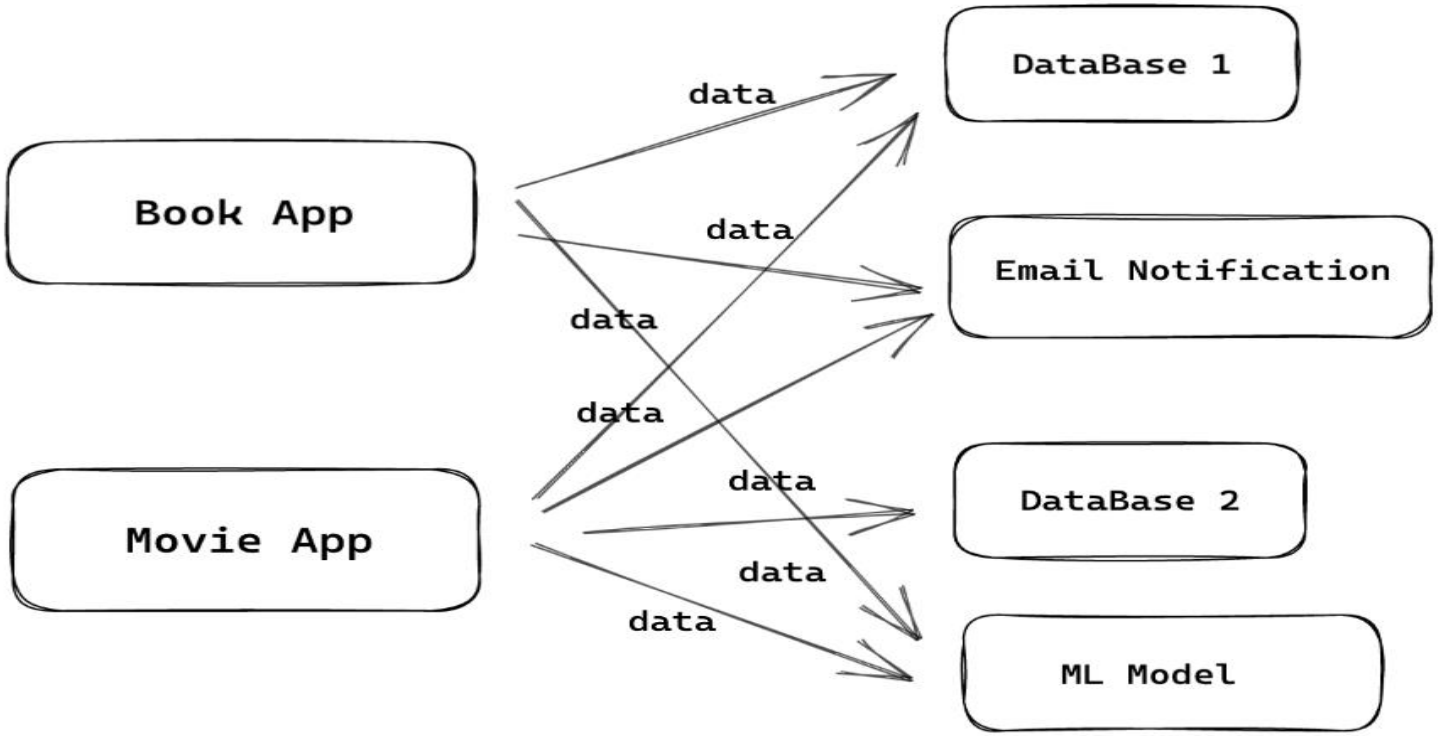
Use case with Kafka

- Book Rental App – When user places an order, system saves a new row in a database and sends a confirmation email. looks something like this:



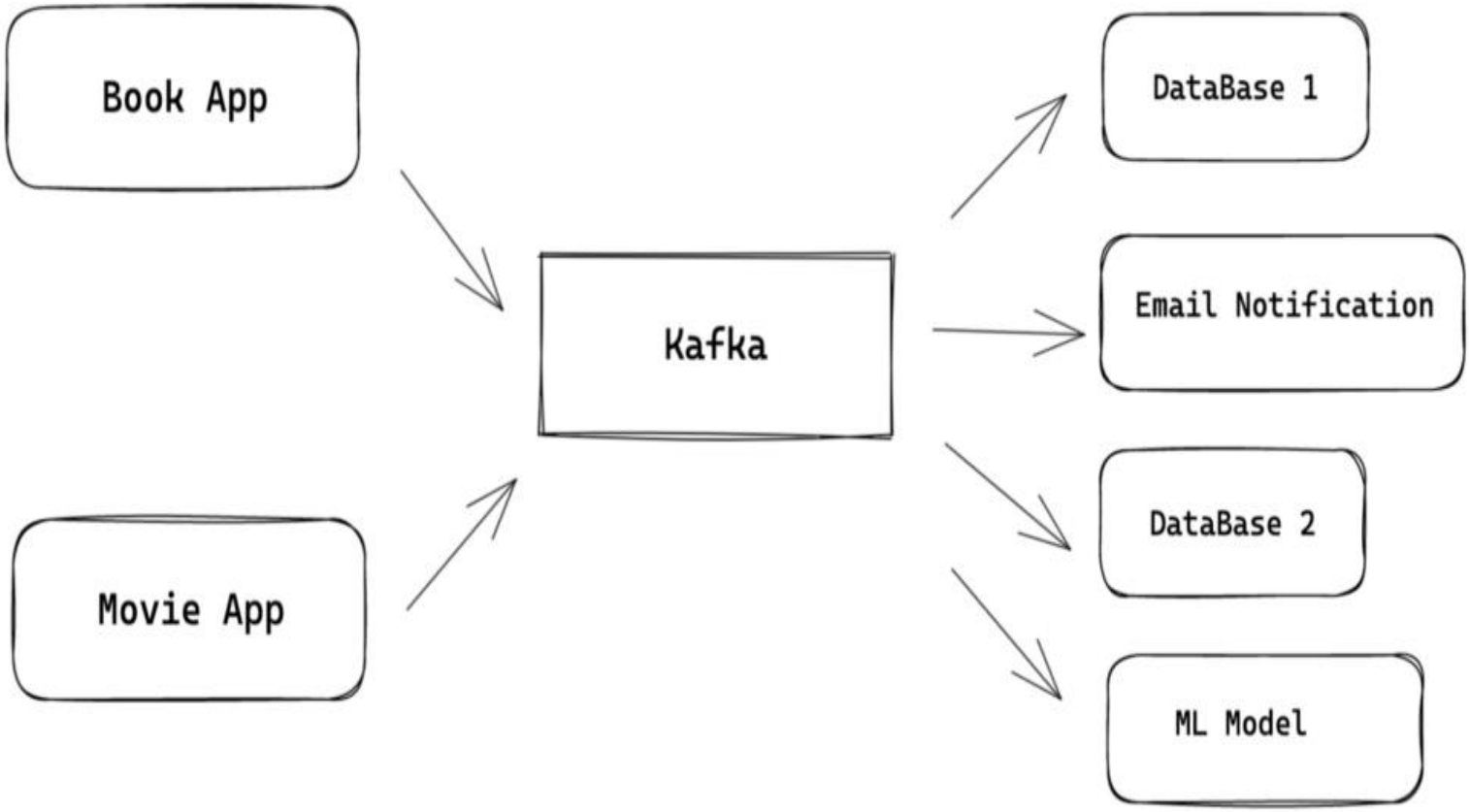
contd..

- **Add a new product :** Movie app. Now, if a user rents a movie, you want to keep track of it in a different database.
- Also add a Machine Learning model to create personalised recommendations.
- A look at system design and you find this:



contd..

- Kafka can bring some order. Something like this:



Kafka API

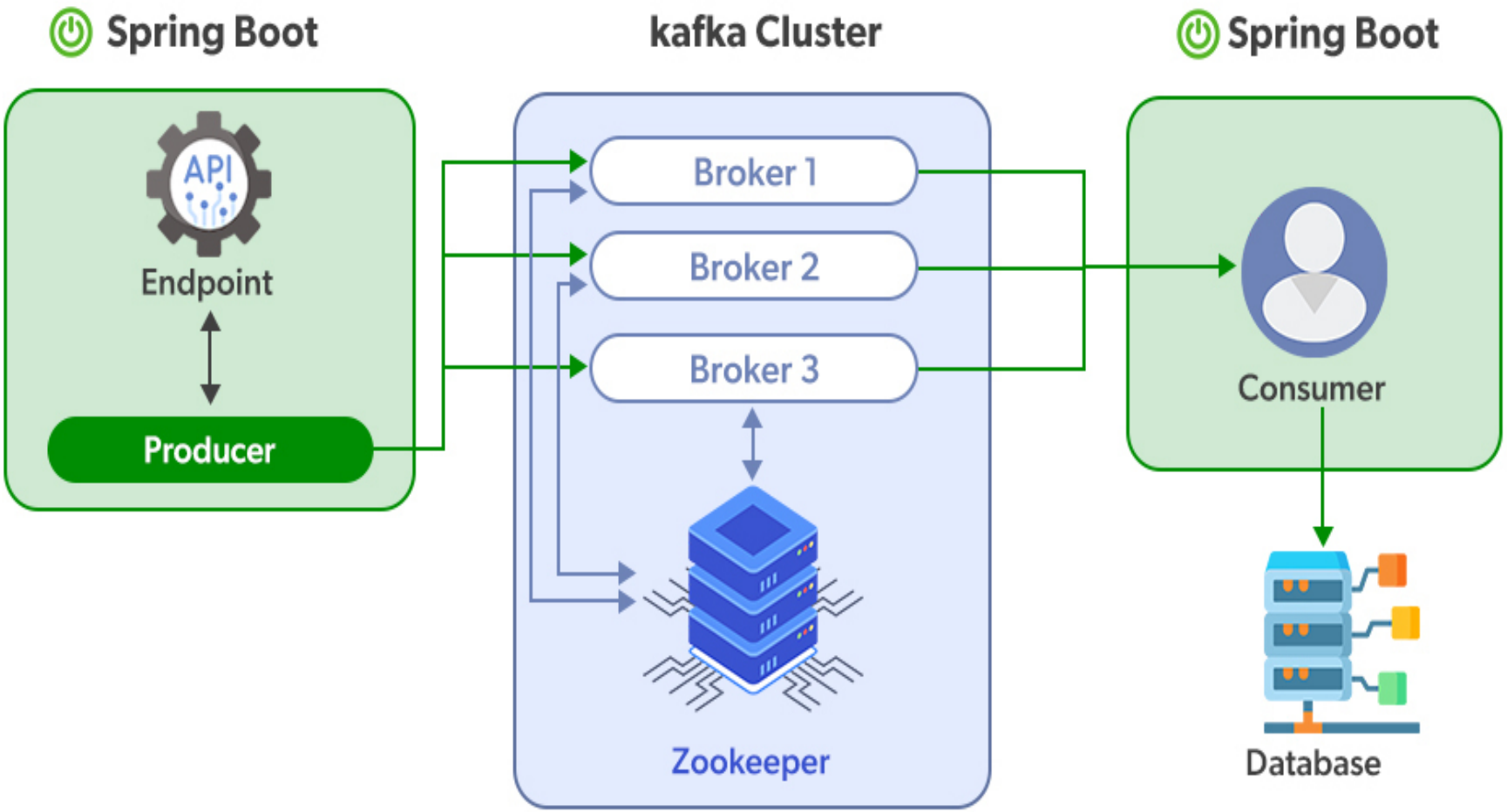
- In Apache Kafka, these classes are fundamental components of the client APIs used to produce (write) and consume (read) data.
- **Producer Components**
  - ✓ **ProducerConfig**: class or a Properties object, used to hold **configuration settings** for a KafkaProducer instance. Key config include **bootstrap.servers** (to connect to the cluster), **key.serializer, value.serializer** (to convert Java objects into bytes for transmission), and acks (to set data durability guarantees).
  - ✓ **KafkaProducer**: is the main **client class** that application uses to **publish (write)** records to Kafka topics. Manages a buffer of records and provides methods e.g. send() to asynchronously send **Producer Record** objects to appropriate Kafka broker and partition.
  - ✓ **ProducerRecord**: represents single **message (event)** that is intended to be sent to Kafka. It is a **key/value** pair and requires a target topic name. It can also **optionally specify** a **partition** and a **key**, which determines which partition the message will be written to for guaranteed ordering by key.

contd..

- **Consumer Components**

- ✓ **ConsumerConfig**: the class (or Properties object) defines the **config properties** for a KafkaConsumer. Key settings are **bootstrap.servers**, **group.id** (to assign the consumer to a consumer group for coordinated consumption), **key.deserializer**, **value.deserializer** (to convert bytes back into Java objects), and **enable.auto.commit** (for managing offset commits).
- ✓ **KafkaConsumer**: the main **client class** for application to **subscribe** to **topics** and **consume** (read and process) streams of records from the Kafka cluster. It handles group coordination, partition assignment, and offset management, allowing applications to efficiently retrieve data using the poll() method.
- ✓ **ConsumerRecord**: represents a single **record** that has been **received** from Kafka cluster after a KafkaConsumer.poll() operation. It contains the **message's key & value**, along with metadata such as the topic name, the partition number it came from, and its offset within that partition.

Kafka with SpringBoot



Kafka with SpringBoot

- @EnableKafka
- @KafkaListener
- ConsumerFactory
- ConcurrentKafkaListenerContainerFactory
- KafkaTemplate<String, String>

Kafka Metadata

- Metadata storage in Apache Kafka refers to the management of stateful information that defines the structure and health of the Kafka cluster. This metadata includes information about topics, partition locations, broker status, consumer group offsets, and access control lists (ACLs).
- Effective metadata management is crucial for Kafka's functionality; if it becomes unavailable, the system cannot elect leaders, assign replicas, or coordinate consumers.
- Metadata Storage Approaches
- Kafka provides two main ways to store this information:
- KRaft (Kafka Raft Metadata Mode): This is the modern, recommended approach (default as of newer Kafka versions).
- Mechanism: Metadata is stored in a dedicated internal topic called __cluster_metadata.

contd..

- Structure: It is a single-partition, replicated log (using the Raft consensus protocol) among Kafka controllers.
- Benefit: It removes the need for external dependencies like ZooKeeper, allowing Kafka to scale to millions of partitions with faster recovery.
- Apache ZooKeeper: This is the legacy approach.
- Mechanism: Metadata is organized into a hierarchy of ZNodes.
- Status: Support is being phased out, with plans for full removal in future Kafka releases.
- Key Components of Metadata
- Kafka metadata keeps track of the "what, where, when, how, and who" of the data system:
- Topic Configuration: Names, partition counts, replication factors, and retention policies.

contd..

- Broker Information: Which brokers are active in the cluster.
- Partition Leadership: Which broker is currently the leader for a specific topic-partition.
- In-Sync Replicas (ISRs): Which replicas are currently synchronized with the leader.
- Consumer Offsets: Position tracking for consumer groups.
- ACLs: Security and access control policies.
- Offset Metadata Storage
- In addition to cluster-level metadata, Kafka allows consumers to store custom, context-rich metadata alongside the offsets they commit.
- Purpose: To attach stateful information (e.g., custom timestamps, process IDs, or aggregated state) to a specific message, helping with troubleshooting or complex processing scenarios.
- Functionality: This metadata is stored when a consumer marks a message as consumed (mark_as_consumed!).

contd..

- Tiered Storage Metadata
- When using Tiered Storage, Kafka stores extra metadata to manage data moved to external object storage (e.g., S3).
- remote_log_metadata topic: An internal topic used to store metadata about remote log segments, such as where data is located in external storage and its state (e.g., COPY_STARTED or COPY_FINISHED)

KRaft

- KRaft (Kafka Raft) mode in Apache Kafka was introduced as a preview in version 2.8.0, deemed production-ready for new clusters in version 3.3.1, and becomes the default (replacing ZooKeeper) in Kafka 4.0.
- Key KRaft Version Milestones:
  - ✓ 2.8.0: Initial preview, allowed testing without ZooKeeper.
  - ✓ 3.0.0: Significant improvements and stabilization.
  - ✓ 3.3.1+: Recommended version to use as "production ready".
  - ✓ 3.7.0: Stable, fully-featured, and commonly used for migration in Amazon MSK.
  - ✓ 4.0.0: ZooKeeper is removed entirely; KRaft is mandatory.
- For new deployments, it is highly recommended to use the latest 3.x or 4.x versions to leverage built-in quorum management.